AdvR 02 – Vectors

Atomic Vectors

Vector Types

Two flavors of vectors in R:

1. **Atomic vectors** — all elements same type
2. **Lists** — elements can be any type

Four common atomic types:

Type	Example	`typeof()`
Logical	`TRUE`, `FALSE`	`"logical"`
Integer	`1L`, `6L`	`"integer"`
Double	`1.5`, `1.23e4`	`"double"`
Character	`"hello"`	`"character"`

Two rare types: `complex` and `raw`.

"Numeric" = integer + double collectively.

Creating Vectors with c()

```
lgl_var <- c(TRUE, FALSE)
int_var <- c(1L, 6L, 10L)
dbl_var <- c(1, 2.5, 4.5)
chr_var <- c("these are", "some strings")

typeof(int_var)
```

```
#> [1] "integer"
```

```
length(dbl_var)
```

```
#> [1] 3
```

`c()` **flattens** nested calls:

```
c(c(1, 2), c(3, 4))
```

```
#> [1] 1 2 3 4
```

Special doubles: `Inf`, `-Inf`, `NaN`.

Missing Values

`NA` is infectious — most operations with `NA` return `NA`:

```
NA > 5
```

```
#> [1] NA
```

```
10 * NA
```

```
#> [1] NA
```

Exceptions (identity holds for all inputs): `NA ^ 0` → `1`, `NA | TRUE` → `TRUE`, `NA & FALSE` → `FALSE`.

Test with `is.na()`, never `==`. Logical → numeric is useful:

```
x <- c(FALSE, FALSE, TRUE, TRUE, FALSE)
sum(x)      # count TRUEs
```

```
#> [1] 2
```

```
mean(x)     # proportion of TRUEs
```

```
#> [1] 0.4
```

Four typed NAs: `NA` (logical), `NA_integer_`, `NA_real_`, `NA_character_`.

Coercion

When types are mixed, R coerces to the most flexible type:

$$\text{logical} \rightarrow \text{integer} \rightarrow \text{double} \rightarrow \text{character}$$

```
c(1, FALSE)         # double: c(1, 0)
```

```
#> [1] 1 0
```

```
c("a", 1)           # character: c("a", "1")
```

```
#> [1] "a" "1"
```

```
c(TRUE, 1L)         # integer: c(1L, 1L)
```

```
#> [1] 1 1
```

Manual coercion with `as.*()`:

```
as.integer("42")
```

```
#> [1] 42
```

```
as.double(TRUE)
```

```
#> [1] 1
```

Test with `is.na()`, not `== NA` (which returns `NA`).

Exercises (Atomic Vectors)

1. How do you create raw and complex scalars? (Hint: `?raw` and `?complex`.)
2. Predict the output:

```
c(1, FALSE)
c("a", 1)
c(TRUE, 1L)
```

3. Why is `1 == "1"` true? Why is `-1 < FALSE` true? Why is `"one" < 2` false?
4. Why is the default missing value `NA` a logical vector? (Hint: think about `c(FALSE, NA_character_)`.)

Solutions (Atomic Vectors)

1. `as.raw(42)` or `charToRaw("A")` for raw. `complex(real = 1, imaginary = 1)` or `1 + 1i` for complex.
2.

```
str(c(1, FALSE))      # double: c(1, 0)
```

```
#>  num [1:2] 1 0
```

```
str(c("a", 1))        # character: c("a", "1")
```

```
#>  chr [1:2] "a" "1"
```

```
str(c(TRUE, 1L))      # integer: c(1L, 1L)
```

```
#>  int [1:2] 1 1
```

3. Comparison operators coerce to common type. `1` → `"1"`, `FALSE` → 0, `2` → `"2"` (and `"one" > "2"` lexicographically).
4. Logical is the least flexible type. `NA` coerces **up** (to `NA_integer_`, etc.) without affecting the result type. If `NA` were character, everything would become character.

Attributes

Getting and Setting Attributes

Attributes are **name-value pairs** attached to any object:

```
a <- 1:3
attr(a, "x") <- "abcdef"
attr(a, "y") <- 4:6
str(attributes(a))
```

```
#> List of 2
#>  $ x: chr "abcdef"
#>  $ y: int [1:3] 4 5 6
```

Or use `structure()`:

```
a <- structure(1:3, x = "abcdef", y = 4:6)
str(attributes(a))
```

```
#> List of 2
#>  $ x: chr "abcdef"
#>  $ y: int [1:3] 4 5 6
```

Most attributes are lost by operations:

```
attributes(a[1])   # NULL
```

```
#> NULL
```

Two exceptions routinely preserved: **names** and **dim**.

Names and Dimensions

Names — three ways to create:

```
x <- c(a = 1, b = 2, c = 3)          # at creation
names(x) <- c("a", "b", "c")          # assignment
x <- setNames(1:3, c("a", "b", "c")) # setNames()
```

Remove with `names(x) <- NULL` or `unname(x)`.

Dimensions turn vectors into matrices/arrays:

```
x <- matrix(1:6, nrow = 2, ncol = 3)
x
```

```
#>      [,1] [,2] [,3]
#> [1,]    1    3    5
#> [2,]    2    4    6
```

Vector	Matrix	Array
`names()`	`rownames()`, `colnames()`	`dimnames()`
`length()`	`nrow()`, `ncol()`	`dim()`
`c()`	`rbind()`, `cbind()`	`abind::abind()`

Exercises (Attributes)

1. How is `setNames()` implemented? How is `unname()` implemented?
2. What does `dim()` return for a 1-d vector? When might you use `NROW()` or `NCOL()`?
3. Describe these objects. What makes them different from `1:5`?

```
x1 <- array(1:5, c(1, 1, 5))
x2 <- array(1:5, c(1, 5, 1))
x3 <- array(1:5, c(5, 1, 1))
```

Solutions (Attributes)

1. `setNames()` assigns `names(object) <- nm` and returns the object. `unname()` sets `names(obj) <- NULL` (and optionally `dimnames`).
2. `dim()` returns `NULL` for a plain vector. Use `NROW()`/`NCOL()` to treat vectors uniformly with matrices:

```
x <- 1:10
c(nrow(x), ncol(x))      # NULL, NULL
```

```
#> NULL
```

```
c(NROW(x), NCOL(x))      # 10, 1
```

```
#> [1] 10  1
```

3. All three are 3-d arrays with a `dim` attribute (1-by-1-by-5, 1-by-5-by-1, 5-by-1-by-1). Unlike `1:5`, they have dimensionality.

S3 Atomic Vectors

Factors

Factors store categorical data. Built on integer vectors with `levels`:

```
x <- factor(c("a", "b", "b", "a"))
typeof(x)
```

```
#> [1] "integer"
```

```
levels(x)
```

```
#> [1] "a" "b"
```

Factors record **all possible** values, even unobserved:

```
sex_char <- c("m", "m", "m")
sex_factor <- factor(sex_char, levels = c("m", "f"))
table(sex_char)
```

```
#> sex_char
#> m
#> 3
```

```
table(sex_factor)
```

```
#> sex_factor
#> m f
#> 3 0
```

Dates and Date-Times

Dates — doubles representing days since 1970-01-01:

```
today <- Sys.Date()
typeof(today)
```

```
#> [1] "double"
```

```
unclass(as.Date("1970-02-01"))  # 31 days
```

```
#> [1] 31
```

Date-times (POSIXct) — doubles representing seconds since epoch:

```
now_ct <- as.POSIXct("2018-08-01 22:00", tz = "UTC")
typeof(now_ct)
```

```
#> [1] "double"
```

Durations (difftime) — doubles with a `units` attribute:

```
as.difftime(1, units = "weeks")
```

```
#> Time difference of 1 weeks
```

Exercises (S3 Vectors)

1. What sort of object does `table()` return? What is its type? What attributes does it have?
2. What happens to a factor when you modify its levels?

```
f1 <- factor(letters)
levels(f1) <- rev(levels(f1))
```

3. How do `f2` and `f3` differ from `f1`?

```
f2 <- rev(factor(letters))
f3 <- factor(letters, levels = rev(letters))
```

Solutions (S3 Vectors)

1. `table()` returns an integer array with class `"table"`, a `dim` attribute, and `dimnames`. Dimensions correspond to unique values in each input.
2. The **underlying integers stay the same** but the level labels change. This makes it *look* like the data changed, but only the mapping changed:

```
f1 <- factor(letters[1:5])
as.integer(f1)
```

```
#> [1] 1 2 3 4 5
```

```
levels(f1) <- rev(levels(f1))
as.integer(f1)   # still 1 2 3 4 5
```

```
#> [1] 1 2 3 4 5
```

```
f1               # but displays as e d c b a
```

```
#> [1] e d c b a
#> Levels: e d c b a
```

3. `f2 <- rev(factor(letters))` reverses **element order** (integers change, levels stay). `f3 <- factor(letters, levels = rev(letters))` reverses **level order** (integers change, levels reversed).

Lists

Lists: Heterogeneous Vectors

Each element can be **any type**, including other lists:

```
l1 <- list(1:3, "a", c(TRUE, FALSE), c(2.3, 5.9))
str(l1)
```

```
#> List of 4
#>  $ : int [1:3] 1 2 3
#>  $ : chr "a"
#>  $ : logi [1:2] TRUE FALSE
#>  $ : num [1:2] 2.3 5.9
```

Lists store **references** (not copies) and are **recursive**:

```
lobstr::obj_size(mtcars)
```

```
#> 7.21 kB
```

```
lobstr::obj_size(list(mtcars, mtcars, mtcars))
```

```
#> 7.29 kB
```

Three copies of `mtcars` in a list costs only ~80 extra bytes. Lists of lists: `list(list(list(1)))`.

Combining and Coercing Lists

`c()` with lists combines them — `list()` nests, `c()` flattens:

```
str(list(list(1, 2), c(3, 4)))   # nested
```

```
#> List of 2
#>  $ :List of 2
#>   ..$ : num 1
#>   ..$ : num 2
#>  $ : num [1:2] 3 4
#> ... [output truncated]
```

```
str(c(list(1, 2), c(3, 4)))      # flat
```

```
#> List of 4
#>  $ : num 1
#>  $ : num 2
#>  $ : num 3
#>  $ : num 4
#> ... [output truncated]
```

`as.list()` converts atomic to list. `unlist()` does the reverse.

Exercises (Lists)

1. List all ways a list differs from an atomic vector.
2. Why do you need `unlist()` to convert a list to an atomic vector? Why doesn't `as.vector()` work?
3. Compare `c()` and `unlist()` when combining a date and a date-time.

Solutions (Lists)

1. Key differences:
   - **Heterogeneous** vs. homogeneous types
   - Elements are **references** to separate objects
   - Can be **recursive** (lists of lists)
   - Out-of-bounds subsetting returns `NULL`, not `NA`
2. A list **is already a vector** (just not atomic). `as.vector()` won't change that. `unlist()` recursively extracts and coerces elements into an atomic vector.
3. `c()` converts to a common S3 type (POSIXct):

```
c(as.Date("1970-01-02"),
  as.POSIXct("1970-01-01 01:00", tz = "UTC"))
```

```
#> [1] "1970-01-02" "1970-01-01"
```

`unlist()` strips **all attributes** — returns raw doubles (losing date meaning).

Data Frames and Tibbles

Data Frame Basics

Data frames are **named lists of equal-length vectors**:

```
df <- data.frame(x = 1:3, y = c("a", "b", "c"))
typeof(df)
```

```
#> [1] "list"
```

```
attributes(df) |> str()
```

```
#> List of 3
#>  $ names    : chr [1:2] "x" "y"
#>  $ class    : chr "data.frame"
#>  $ row.names: int [1:3] 1 2 3
```

Key properties:

```
names(df)
```

```
#> [1] "x" "y"
```

```
nrow(df)
```

```
#> [1] 3
```

```
ncol(df)
```

```
#> [1] 2
```

Tibbles vs. Data Frames

Tibbles are a modern reimagining with stricter behavior:

Feature	`data.frame`	`tibble`
String coercion	Converts to factor (pre-4.0)	Never
Non-syntactic names	Transforms	Preserves
Recycling	Integer multiples	Length-1 only
`$` partial matching	Yes (silent)	No (warns)
`[, "col"]` returns	Vector	Tibble

```
names(data.frame(`1` = 1))  # "X1" (transformed)
```

```
#> [1] "X1"
```

```
names(tibble(`1` = 1))      # "1" (preserved)
```

```
#> [1] "1"
```

Tibbles allow **column references** during creation: `tibble(x = 1:3, y = x * 2)`.

Row Names and List Columns

Row names are discouraged — use a column instead:

```
df3 <- data.frame(age = c(35, 27),
  row.names = c("Bob", "Sue"))
as_tibble(df3, rownames = "name")
# Converts row names to a regular column
```

List columns store complex data per row:

```
tibble(x = 1:3, y = list(1:2, 1:3, 1:4))
```

```
#> # A tibble: 3 x 2
#>       x y
#>   <int> <list>
#> 1     1 <int [2]>
#> 2     2 <int [3]>
#> ... [output truncated]
```

In base R, list columns need `I()`: `data.frame(x = 1:3, y = I(list(...)))`.

Exercises (Data Frames)

1. Can you have a data frame with zero rows? Zero columns?
2. What happens if you set rownames that are not unique?
3. If `df` is a data frame, what can you say about `t(df)` and `t(t(df))`?
4. What does `as.matrix()` do with mixed column types? How does `data.matrix()` differ?

Solutions (Data Frames) — 1–2

1. Yes to both:

```
data.frame(a = integer(), b = logical()) # zero rows
```

```
#> [1] a b
#> <0 rows> (or 0-length row.names)
```

```
data.frame()                             # zero columns
```

```
#> data frame with 0 columns and 0 rows
```

```
mtcars[0, 0]                             # both zero
```

```
#> data frame with 0 columns and 0 rows
```

2. Data frames **error** on duplicate row names. But subsetting with `[` can create duplicates (auto-deduplicated with suffixes).

```
df <- data.frame(x = 1:3)
row.names(df) <- c("x", "y", "y")
```

```
#> Error in `.rowNamesDF<-`(x, value = value): duplicate 'row.names' are not allowed
```

3. `t(df)` coerces to matrix first (`as.matrix()`), then transposes. With mixed types, everything becomes the most flexible type (usually character). `t(t(df))` transposes back but remains a matrix.

```
df <- data.frame(x = 1:3, y = letters[1:3])
t(df)  # character matrix
```

```
#>   [,1] [,2] [,3]
#> x "1"  "2"  "3"
#> y "a"  "b"  "c"
```

4. `as.matrix()` coerces to the most flexible type (character if any character columns). `data.matrix()` always returns a **numeric** matrix — factors become integer codes, characters become `NA`.

NULL

NULL: The Empty/Absent Vector

`NULL` is a special type of length zero with no attributes:

```
typeof(NULL)
```

```
#> [1] "NULL"
```

```
length(NULL)
```

```
#> [1] 0
```

Two uses: (1) **empty vector** (`c()` returns `NULL`) and (2) **absent vector** (default argument when no value makes sense).

Contrast with `NA`:

- `NULL` = the **vector** is absent (disappears in `c()`)
- `NA` = an **element** is absent (stays in `c()`)

```
c(NULL, 1, 2)      # NULL disappears
```

```
#> [1] 1 2
```

```
c(NA, 1, 2)        # NA stays
```

```
#> [1] NA  1  2
```